OFICINA · 2 DIAS · 6 HORAS

Do Código à Produção
DevOps integrado à Inteligência Artificial na Prática

FELIPE S.R. MACEDO



VISÃO GERAL

O que você vai saber fazer

6h

de oficina prática

2 dias × 3 horas

Dia 1: fundamentos
Dia 2: mão na massa

01 **Usar uma VM Linux**
configurar o ambiente e criar ambientes de teste

02 **Usar APIs, MCP e RPC**
entender como sistemas e IAs conversam entre si

03 **Containerizar a aplicação**
Docker e Docker Compose, com segredos fora do código

04 **Montar uma pipeline de CI/CD**
GitHub Actions com portão de qualidade

05 **Usar IA no DevOps**
acelerar tarefas sem abrir mão da revisão humana

COMO A OFICINA FUNCIONA

Teoria, tarefas e um projeto real



Aula teórica

Os conceitos por trás de cada etapa, com demos ao vivo no terminal



Tarefas

Cada bloco termina num checkpoint: quem travar pula para a branch pronta



Projeto no GitHub

A API Suporte Bot rodando em produção, com CI verde e URL pública

Suporte Bot: uma API que responde dúvidas de clientes usando um LLM — o fio condutor dos dois dias.

ETAPAS

O roteiro dos dois dias

01
DevOps
a cultura

02
Git e GitHub
trabalho em equipe

03
Documentação
o que ninguém lembra

04
Linux
o básico do servidor

05
VM e VPS
onde o código roda

06
Ambientes
testes × produção

07
Chaves de API
acesso e custo

08
Integração de IA
MCP na prática

O que é DevOps?

Development + Operations: uma cultura que aproxima quem escreve o código de quem o mantém rodando.

DEVELOPMENT

Planejar · Codificar · Testar

+

OPERATIONS

Implantar · Monitorar · Manter

O QUE UM DEVOPS FAZ



Automatiza testes



Infraestrutura como código (IaC)



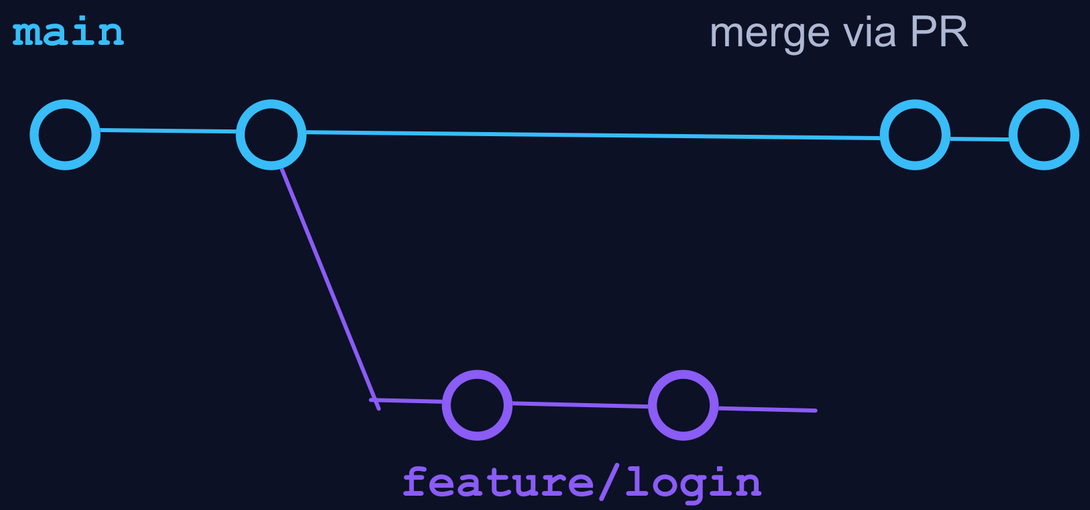
Cuida da base de testes



Controla o que sobe para produção

Git guarda a história do código

Cada commit é uma foto do projeto. Branches deixam várias pessoas trabalharem sem pisar no código umas das outras.



bash — git

```
# começar a trabalhar
$ git clone <repo>
$ git switch -c feature/login
# salvar o progresso
$ git add .
$ git commit -m "login por chave"
$ git push origin feature/login
# depois: abrir o PR no GitHub
```

Trabalhar em equipe com pull requests



O que nunca vai para o repositório

`.env e chaves` `senhas e tokens` `node_modules` `arquivos de log`

Liste tudo isso no .gitignore antes do primeiro commit.

CI e CD: do commit ao cliente

CI
Integração Contínua

A cada push, o código é juntado ao dos colegas, testado e empacotado automaticamente.

Pergunta que responde: o código funciona?

CD
Entrega Contínua

Toda versão que passa na CI fica pronta para produção. Alguém aperta o botão de publicar.

Pergunta que responde: está pronto para ir?

CD
Deploy Contínuo

Passou na CI, vai direto para produção. Nenhum clique, nenhuma pessoa no caminho.

Pergunta que responde: já está no ar?

`push` `lint` `testes` `build` `imagem` `staging` `produção`

CI cobre do push à imagem · **CD** leva a imagem até o cliente

CI/CD NO MERCADO

Não é diferencial: é pré-requisito

71%

dos devs usam Docker

+17 pontos em um ano: o maior salto da pesquisa

Stack Overflow Survey 2025

92%

dos profissionais de TI usam containers

eram 80% no ano anterior

Docker State of App Dev 2025

#1

GitHub Actions

ferramenta de CI/CD mais citada (40%), à frente de GitLab e Jenkins

Docker State of App Dev 2025

90%

dos devs já usam IA

e a IA amplifica times com CI/CD sólido, não substitui

DORA 2025 (Google Cloud)

COMO APARECE NAS VAGAS

- ✓ pipelines de CI/CD (GitHub Actions, GitLab CI, Jenkins)
- ✓ Docker, Linux e deploy em ambientes de teste e produção
- ✓ uso ativo de ferramentas de IA no desenvolvimento

Para o júnior: um repo com CI verde e deploy no ar prova mais que um certificado.

Se não está no README, não existe

A documentação mora junto do código e muda junto com ele. Um bom README responde quatro perguntas:

O que é?

o problema que o projeto resolve

Como rodo?

passo a passo do zero até funcionar

O que configuro?

variáveis de ambiente, sem os valores secretos

Como publico?

deploy, rollback e quem chamar

README.md

```
# Suporte Bot

## Como rodar
$ cp .env.example .env
$ docker compose up

## Variáveis
LLM_API_KEY=  # no .env
```

IA rascunha, você revisa e assina embaixo.

Seis coisas para sobreviver no servidor

Terminal

```
$ pwd · ls · cd
```

Arquivos

```
$ cat · nano · cp · mv
```

Permissões

```
$ chmod · chown · sudo
```

Pacotes

```
$ sudo apt install nginx
```

Serviços

```
$ systemctl status docker
```

SSH

```
$ ssh deploy@<ip-da-vm>
```

Protegendo a porta de entrada

SSH com chave
sem login por senha

Usuário sem root
sudo só quando precisa

Firewall (ufw)
só as portas necessárias

Fail2Ban
bane quem erra login demais

Atenção na oficina

A turma inteira sai pelo mesmo IP. Quem errar a senha algumas vezes bane a sala toda.

fail2ban

```
# jail.local
ignoreip = <ip-da-sala>
bantime  = 10m
$ sudo fail2ban-client
    status sshd
```

Onde o código vai morar



VM local

Um computador dentro do seu computador

- VirtualBox, Multipass
- grátis, ótima para treinar
- desliga quando você desliga



VPS

Uma VM alugada num data center

- IP público, 24h no ar
- custo mensal fixo
- é aqui que fica a produção

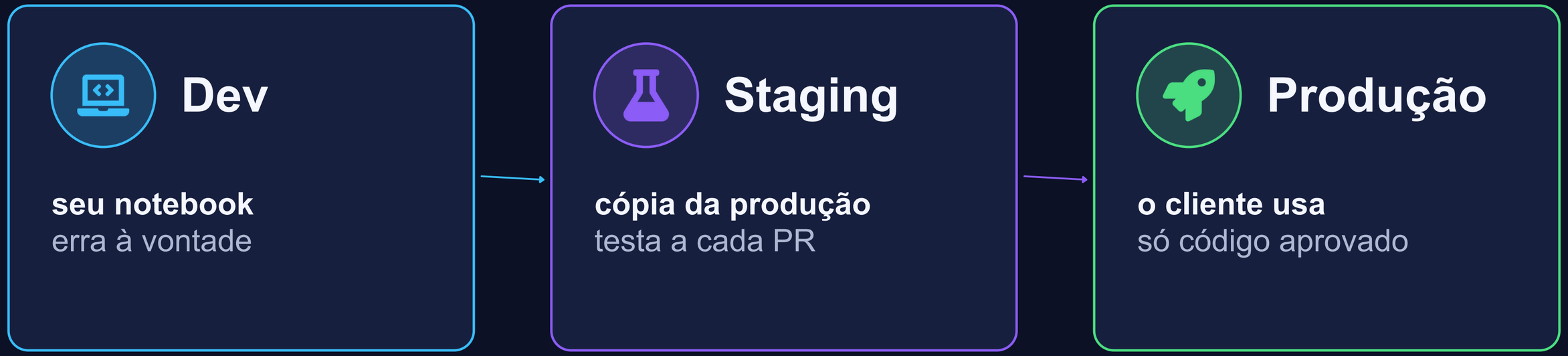


Container

Só a aplicação e o que ela precisa

- roda dentro da VM ou VPS
- sobe em segundos
- igual em todo ambiente

Teste antes de quebrar a produção



Mesma imagem, configuração diferente
o que muda entre ambientes vem do .env

Cada ambiente com suas chaves
staging nunca usa a chave da produção

Rollback pronto
voltar para a imagem anterior em um comando

A chave é a sua identidade (e o seu cartão)



requisição HTTP

```
POST /v1/messages
Authorization: Bearer sk-••••

# a API confere a chave,
# aplica limites e cobra a conta
```

Guarde no .env
e nos secrets do GitHub, nunca no código

Limite de gasto
toda chamada custa: defina um teto

Uma chave por ambiente
e por pessoa, para saber quem usou

Vazou? Revogue
gere outra na hora, não espere

MCP: a porta USB das IAs

O Model Context Protocol padroniza como uma IA descobre e usa ferramentas de outros sistemas. Por baixo, é JSON-RPC 2.0: JSON indo e voltando.



mensagem MCP

```
{"jsonrpc": "2.0", "id": 7,
 "method": "tools/call",
 "params": {"name": "abrir_chamado",
   "arguments": {"cliente": "4021"}}}
```

AO VIVO NO DIA 2

O Claude consulta a conexão de um cliente e abre um chamado em produção, na frente da turma.

Amanhã a gente coloca tudo em produção



PARA TRAZER PRONTO

Docker · Git configurado · conta no GitHub · chave SSH · modelo do Ollama

FIM DO DIA 1

Perguntas?

Do código à produção — com IA do lado.

FELIPE S.R. MACEDO